ChatGPT

# Efficient Transformer Design for Long Contexts and Fast Inference

## 1. Introduction: Long Contexts and Efficiency Challenges

Transformers have revolutionized sequence modeling, but handling very long input contexts (thousands of tokens) poses significant efficiency challenges. In training, attention is computed in parallel over the full sequence, but at inference time (especially autoregressive generation), processing long contexts becomes **memory and bandwidth-bound** rather than compute-bound [1] [2]. Each new token requires loading and using all key/value pairs from previous tokens, leading to heavy memory traffic. For example, generating with an 8k-token context in a 32-head Transformer can involve repeatedly accessing gigabytes of key/value data, which becomes a bottleneck on modern hardware [1] [3]. Storing all past keys/values ("KV caching") avoids recomputation but consumes memory linear in sequence length *and* model size [4]. If the cache grows beyond GPU memory, it must be offloaded to CPU or disk at great cost in speed [3].

The motivation for **long-context modeling** is clear: tasks like summarization of long documents, code completion, or dialogue with extensive history all benefit from longer memory. Recent large language models such as Anthropic's Claude can handle up to 100k tokens, enabling analysis of entire books or multi-hour transcripts in one go. However, achieving such context lengths demands innovations in model architecture and inference strategy. Without careful design, **latency** (per-token delay) and **throughput** (tokens processed per second) degrade sharply as context length increases. The quadratic time/space complexity of attention is the primary culprit, along with the memory bandwidth limits mentioned above.

In this guide, we explore **efficient Transformer design techniques** that enable long context handling and faster inference. We will cover modern positional encodings for extrapolation, architectural changes to the attention mechanism, normalization improvements, and caching strategies. Concepts will be explained both at a high level and with some PyTorch implementation examples. We also include real benchmark data (e.g. latency and memory usage at 8k context) and case studies of how these ideas manifest in real-world models like LLaMA, Claude, and PaLM. By the end, you should understand how to modify Transformer models to **efficiently handle long sequences** and what trade-offs are involved.

**Key challenges for long-context Transformers include:**

- **Positional encoding limits:** Many models train on 1–2k token sequences and use absolute position embeddings that don't generalize to longer sequences. We need methods like RoPE or ALiBi that allow **extrapolation** beyond training lengths [5] [6].
- **Quadratic attention cost:** Self-attention on sequences of length $N$ is $O(N^2)$ in compute and memory. Even storing past keys/values for reuse can overwhelm memory at long $N$. We will see how **multi-query attention (MQA)** and **grouped-query attention (GQA)** reduce memory bandwidth and storage by sharing keys/values across heads [7] [8].

- **Normalization and numerical stability:** Techniques like **RMSNorm** (Root Mean Square Norm) can replace LayerNorm to reduce computation and improve training stability for very deep models [9] [10].
- **Implementational optimizations:** Efficient caching, mixed precision, and custom kernels (FlashAttention, etc.) can further boost inference speed, though these are beyond the scope of this guide.

By co-designing the model architecture with inference in mind, one can achieve **dramatically faster generation on long sequences**. For instance, using multi-query attention, recent work reported up to **11× higher throughput and 30% lower latency** on long-sequence inference, compared to standard multi-head attention [11] [12]. In the following sections, we delve into the main techniques enabling such improvements.

## 2. Rotary Position Embeddings (RoPE)

Modern large language models often replace traditional positional encodings with **Rotary Position Embeddings (RoPE)** for better length generalization. RoPE, introduced by Su et al. (2021) [13], encodes token positions by applying a deterministic rotation to the query and key vectors in each attention layer, instead of adding fixed position vectors to the inputs. This elegant method infuses relative position information directly into the attention computations.

**Key idea:** RoPE treats each query/key vector as a sequence of 2D coordinates (or complex numbers) and rotates these coordinates by an angle that grows with the position index [14]. Formally, if $\mathbf{x} = (x_0, x_1, x_2, x_3, \dots)$ is the split of a $d$-dimensional vector into pairs, and $\theta$ is a constant factor, then for position $p$ we rotate each pair $(x_{2i}, x_{2i+1})$ by $p\theta$ radians in the plane:

$$ \begin{aligned} u &= x_{2i} \cos(p\theta_i) - x_{2i+1} \sin(p\theta_i),\ v &= x_{2i} \sin(p\theta_i) + x_{2i+1} \cos(p\theta_i), \end{aligned} $$

where $\theta_i$ may be set differently per dimension (e.g. a base angle times a power of 10000 as in sinusoidal encoding). This rotation is applied to **both query $Q$ and key $K$ vectors** at each layer. Crucially, rotating $Q$ and $K$ by the same amount causes their dot product $Q \cdot K$ (which determines attention scores) to depend only on their **relative position difference** [14]. In effect, RoPE encodes positions such that attention naturally incorporates how far apart tokens are, without needing explicit bias terms.

**Comparison to absolute positions:** Unlike learned or sinusoidal absolute position embeddings (which are added to token representations and **don't inherently convey distance**), RoPE is a form of *relative* positioning. The rotation on $Q,K$ means the attention score between token at position $p$ and token at $q$ will include a phase term proportional to $(p-q)$, giving a built-in awareness of distance [14]. RoPE has no learned parameters and can, in principle, extrapolate to positions beyond those seen in training because the rotations can be computed for arbitrary $p$. In practice, RoPE has been found to generalize better to longer sequences than fixed absolute embeddings [15]. It became popular through its use in GPT-Neo and LLaMA (Meta AI's models) – LLaMA removed absolute embeddings and adopted RoPE at all layers [13].

**Implementation:** In code, implementing RoPE involves precomputing rotation matrices (or sin/cos tables) for all needed positions and applying them to the query/key tensors. Below is a simplified PyTorch-like pseudocode for applying RoPE to a batch of queries or keys:

```
def apply_rope(x, cos, sin):
    # x: Tensor of shape [batch, n_heads, seq_len, d], where d is even
    # cos, sin: precomputed tensors of shape [seq_len, d/2] for cosines and
sines
    seq_len = x.size(2)
    d = x.size(3)
    # Split the last dimension into even and odd parts
    x_even = x[..., 0::2]              # shape: [batch, n_heads, seq_len, d/2]
    x_odd  = x[..., 1::2]              # shape: [batch, n_heads, seq_len, d/2]
    # Apply rotation: new_even = x_even * cos(angle) - x_odd * sin(angle)
    #                 new_odd  = x_even * sin(angle) + x_odd * cos(angle)
    cos_angle = cos[:seq_len]          # [seq_len, d/2]
    sin_angle = sin[:seq_len]          # [seq_len, d/2]
    # Expand cos_angle and sin_angle to [1,1,seq_len,d/2] for broadcast
    cos_angle = cos_angle[None, None, :, :]
    sin_angle = sin_angle[None, None, :, :]
    x_rotated_even = x_even * cos_angle - x_odd * sin_angle
    x_rotated_odd  = x_even * sin_angle + x_odd * cos_angle
    # Interleave even and odd dimensions back together
    x_out = torch.stack([x_rotated_even, x_rotated_odd], dim=-1)
    x_out = x_out.view_as(x)  # reshape to [batch, n_heads, seq_len, d]
    return x_out
```

In practice, one would generate `cos` and `sin` from a formula such as $\cos(p \theta_i)$ and $\sin(p \theta_i)$ for each position $p$ and each pair index $i$. The above function would be applied to the query and key tensors after their linear projection but before computing attention scores.

**Why RoPE?** RoPE offers several advantages [16] [17]:

- *Relative encoding:* It naturally encodes relative position differences in the attention mechanism, which helps the model generalize to sequence lengths longer than seen in training.
- *No extra parameters:* The positional information is injected by a fixed trigonometric transform. There are no learned position embeddings, so memory and parameter count don't increase with max length.
- *Extrapolation:* Because it's based on a continuous rotation formula, a model trained on (say) 2k-token sequences can often handle 4k or more at inference, whereas learned absolute embeddings beyond 2k would be uninitialized. (RoPE is not perfect in extrapolation – Press et al. (2021) observed it still degrades for very long inputs, though more gradually than sinusoids [6].)
- *Simplicity:* It's easy to implement and apply at each layer, and it doesn't require modifying the transformer architecture, only the input to the attention calculation.

**Use in practice:** Many state-of-the-art models use RoPE. As mentioned, Meta's **LLaMA** uses RoPE for all layers [13]. Open-source models like GPT-NeoX and Bloom also adopted rotary embeddings to improve robustness on longer texts. The DeepMind **Gopher** and **Chinchilla** models (which still used absolute position embeddings) struggled to extrapolate; newer models like LLaMA showed that RoPE (along with other improvements) could match or surpass those models even at smaller sizes [18] [19]. In summary, RoPE has become a standard positional encoding for large LMs due to its strong performance and minimal downsides.

## 3. ALiBi: Attention with Linear Biases

While RoPE imparts a form of relative positioning, another powerful approach is **Attention with Linear Biases (ALiBi)**, introduced by Press et al. (2021) [5]. ALiBi is a simple method to enable **input length extrapolation** and to bias the model toward recent tokens. Instead of adding any positional vectors to embeddings, ALiBi adds a fixed *bias term* to the attention scores $Q \cdot K^T$ based on the distance between query and key positions [20].

**How ALiBi works:** For each attention head, ALiBi pre-defines a slope (a negative scalar) that is multiplied by the distance (in tokens) between the query position $i$ and key position $j$ (with $i > j$ for causal masking). This product is added to the raw attention score. In effect, every attention score gets a penalty proportional to how far apart the tokens are [20]. The farther away a key is from the query, the more its score is reduced (since the bias is negative). This creates an implicit recency bias – nearer tokens attend more strongly, which intuitively fits language modeling where recent context often matters more.

Importantly, the bias is **linear in distance** (hence "linear biases"), and no new embeddings are introduced. The set of slopes (one per head or one per group of heads) is typically fixed at initialization – Press et al. chose a specific progression of slopes (e.g., geometric sequence) across heads [21]. These biases require no training (though one could learn them, the original method did not).

**Benefits of ALiBi:**

- *Train short, test long:* By not using absolute position embeddings at all and only biasing attention, a model can be trained on, say, 1k-token sequences and **tested on 2k or 4k sequences with minimal perplexity drop** [22]. Press et al. demonstrated a 1.3B-parameter LM trained on 1,024 tokens achieved the *same* perplexity on 2,048-token inputs as a sinusoidal-position model trained on 2,048, simply by using ALiBi [22]. This was accomplished with 11% less training memory and time – effectively ALiBi allows **extrapolation without extra training cost** [22].
- *No additional parameters:* Like RoPE, ALiBi doesn't add embedding vectors. The biases are fixed or use a very small set of learned scalars (in some variants).
- *Compatibility:* It can be applied to any Transformer with minimal code changes – just an adjustment to the attention score matrix after it's computed but before softmax.
- *Empirical performance:* ALiBi was shown to not only extrapolate length, but also sometimes improve performance even at trained lengths due to the inductive bias toward recency [23]. On WikiText-103, ALiBi outperformed several more complex position methods [23].

**Comparison to RoPE and others:** ALiBi's bias is a **monotonic penalty** that effectively *turns off* attention to far-away context in a smooth way. Unlike learned relative position embeddings (as in Transformer-XL or T5's bias), ALiBi's biases are not learned from data but set by a simple rule – this makes it **lightweight and**

**robust**. Press et al. found that sinusoidal absolute embeddings failed to extrapolate (perplexity spiked for longer inputs), RoPE did better but still got worse at 2×–4× training length, whereas ALiBi maintained good performance even up to 10k tokens [24] [25]. Figure 1 below (from their paper) vividly illustrates this:

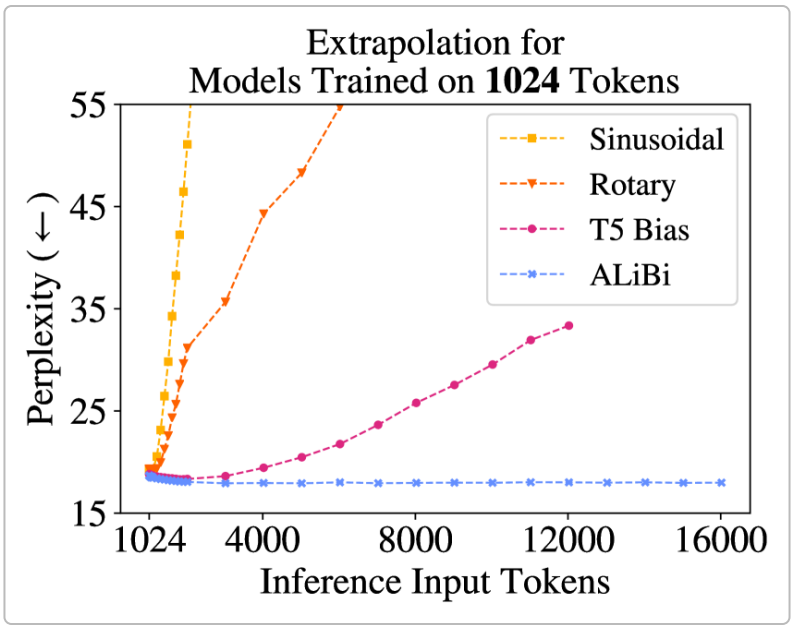



*Figure 1: Perplexity vs. inference sequence length for models trained on 1024 tokens [5] [6]. Absolute sinusoidal embeddings (yellow) fail to extrapolate beyond ~2k (perplexity shoots up). Rotary embeddings (orange) generalize slightly further but then degrade. T5's relative bias (pink) holds out longer but perplexity rises with length. ALiBi (blue dashed) remains flat, indicating stable performance even as input length grows to 16k.* [6] [26]

The reason ALiBi excels at extrapolation is that **the model never "sees" a hard position index** – it only sees *differences*. So there's no concept of "position 1025" being out-of-vocabulary. The attention just gradually weakens for distant tokens, no matter how distant.

**Implementation:** Implementing ALiBi involves:

1. Choosing slopes for each head. Press et al. provide a set of default slopes (e.g., for 16 heads: $[0, -1, -2, -3, \dots]$ scaled appropriately) [21].
2. When computing attention, before the softmax, add the bias: for each query position $i$ and key position $j < i$, add $\text{(slope)} \times (i-j)$ to the score. This can be done by precomputing a distance matrix or by a loop. In practice, one can add a vector of biases to each subsequent position efficiently.
3. Ensure the model is trained with these biases from scratch. The authors found that performance peaks when the eval length is about twice the train length, but even much longer sequences remain usable [22].

**Use cases:** ALiBi is attractive for training long-context models without incurring the full cost of long sequences during training. It was incorporated in some open-source projects (e.g., EleutherAI experimented with ALiBi for GPT-NeoX). If one needs a model to handle, say, 8k tokens but training on 8k is too slow, training on 2–4k with ALiBi and then extrapolating is a viable strategy.

However, ALiBi's recency bias may not be ideal for tasks where very long-range dependencies are **equally important** – it inherently prioritizes closer tokens. In summary, ALiBi is a simple yet powerful positional strategy that enables **"train short, test long"** functionality [5] [27]. It pairs well with models that need to handle varying lengths and has minimal computational overhead.

## 4. Multi-Query and Grouped-Query Attention

Beyond positional encodings, the *attention mechanism itself* can be redesigned for efficiency. Standard multi-head attention uses separate key and value projections for each head. That means if you have $H$ heads, you produce $H$ different key vectors and $H$ value vectors for each token. At inference, all those must be stored and loaded for each generation step. **Multi-Query Attention (MQA)**, proposed by Shazeer (2019), is a variant where **all heads share a single set of key and value vectors** [28]. Essentially, there are multiple query heads as usual, but only *one* "write head" (key/value) for the attention layer [7].

In multi-query attention, instead of $H$ keys and $H$ values per token, we have 1 key and 1 value (or sometimes a small number) per token that all query heads attend to. This dramatically reduces the memory and bandwidth requirements of caching and retrieving keys/values during incremental decoding [7] [8]. **Figure 2** below illustrates the concept in comparison to standard multi-head attention and an intermediate grouped approach:

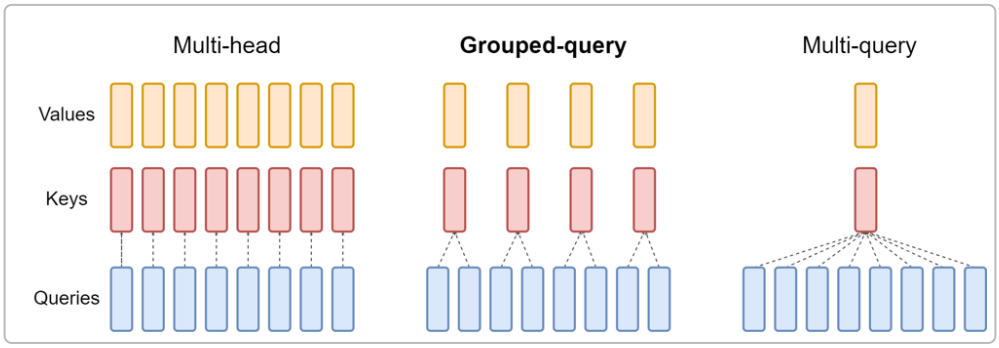



*Figure 2: Schematic of Multi-Head vs. Grouped-Query vs. Multi-Query Attention [29] [30]. In multi-head (left), each of the 8 query heads (blue) has its own key (red) and value (yellow). In multi-query (right), all query heads share one key and one value (dashed lines indicate all queries attending to the single key/value). Grouped-query (middle) is a compromise: here 8 query heads are divided into groups that share keys/values (in this example, 4 groups, so 4 key and value vectors).*

**Multi-Query Attention (MQA):** MQA corresponds to the rightmost part of Figure 2. Concretely, in an MQA layer we have projection matrices: $W^Q$ of shape $d_{\text{model}}\times (H \cdot d_q)$ as usual (each head gets its query), but $W^K$ is $d_{\text{model}}\times d_k$ (not $H \cdot d_k$) and $W^V$ is $d_{\text{model}}\times d_v$. All heads use the single $K$ and $V$. During attention computation, one can broadcast that single key/value across heads so that each query head attends to the same key/value set [31]. The memory saved is a factor of $H$ in key/value storage. **Bandwidth bottleneck reduction:** Instead of loading $H$ keys and $H$ values from memory for each token step, you load 1 – an $H$-fold reduction in memory bandwidth [7] [8]. Shazeer's experiments showed **much faster decoding** as a result, with only a *minor quality degradation* relative to full multi-head attention [32]. In other words, the model might lose a bit of flexibility by not having diverse keys/values per head, but it gains huge speed.

*Why is quality affected?* Intuitively, multi-head attention allows each head to focus on different aspects or positions. With MQA, all heads see the same value information, so the model might not distinguish as finely between different types of content on different heads. Empirically, the drop in accuracy or perplexity is small for moderately sized models but can become more significant in very large models or tasks requiring heterogenous attention patterns [33] . Still, many large production models have accepted this trade-off for the sake of efficiency.

**Grouped-Query Attention (GQA):** Grouped-query attention (the middle of Figure 2) is a generalization introduced by Ainslie et al. (2023) [34] [35] . Instead of 1 or $H$ key-value heads, we choose an intermediate number $G$. The $H$ query heads are divided into $G$ groups, each group sharing one key and value. GQA thus **interpolates between MHA and MQA** [36] [37] :

- If $G = H$, every head has its own key/value (normal multi-head).
- If $G = 1$, all heads share one key/value (multi-query).
- If $1 < G < H$, we have grouped attention.

For example, suppose $H=8$ heads. With $G=2$ groups, you effectively have 2 independent key/value sets, each shared by 4 query heads. With $G=4$, you have 4 key/value sets, each shared by 2 heads, etc. [38] . The memory and speed benefits scale with the reduction in number of key/value sets. If GQA uses $G$ key heads instead of $H$, the KV cache size (and memory bandwidth per step) is reduced by a factor $H/G$ [39] . Ainslie et al. showed that **GQA with a moderate $G$ can recover most of the quality** lost by MQA, while still getting near-MQA speed [34] [40] . In their EMNLP 2023 paper, they fine-tuned existing models to convert to MQA and GQA and found GQA (even with $G$ as low as 8 for a model that originally had 128 heads) maintained *"quality close to multi-head attention with comparable speed to MQA."* [41] .

In practical terms, GQA provides a knob to tune the trade-off between **throughput and accuracy**. One can pick a group count that yields acceptable quality for the application while saving significant memory. As model size increases, the optimal $G$ might shift. Notably, the largest LLaMA-2 models (70B) adopted a form of grouped attention: they use fewer KV heads than query heads (the exact ratio in LLaMA-2 70B is 8 KV groups for 32 query heads, i.e. $G=8$) [42] [43] . This was likely done to mitigate the huge KV cache memory cost at 4k context length for those big models. The Falcon LLM (40B) also used a variant of multi-group attention (one KV per GPU in model parallelism, effectively grouping by partition) [44] . These design choices reflect a trend: **most new large models use MQA or GQA** for faster inference [45] .

**Performance impact:** The impact of MQA/GQA on speed and memory is dramatic especially for long contexts. Because **KV-cache size grows linearly with sequence length**, reducing its size yields more benefit the longer the sequence. In fact, as one analysis noted, *"space savings are proportional to the number of tokens, so [MQA] is particularly beneficial for long sequences."* [12] . A recent benchmark by Fireworks.ai compared a baseline 30B model (without MQA) to a similar model using MQA on long input sequences – the MQA model achieved **11× higher throughput** (tokens/sec) and **~30% lower latency** per token on sequences in the thousands [11] [12] . That is a massive improvement in practical serving capacity.

To put memory numbers in perspective: imagine a model with $H=32$ heads, hidden size $d=4096$. For an 8k token context, a single layer's KV cache (keys+values) for multi-head would require storing $32 \times 8000 \times 4096$ floats for keys (and same for values). That's about **7.8 GB** for one layer's KV【48†】! (Across all layers this is multiplicative, though usually caches are stored per layer.) With MQA, you cut that by 32× to about **0.24 GB** for the same layer – a huge reduction. Even with grouped attention, say $G=4$ (8

heads per group), it'd be one-fourth the size of full MHA. Moreover, the *memory bandwidth* needed to read those keys/values each generation step is similarly reduced, which explains the latency reduction.

The **quality impact** of MQA vs GQA vs MHA has been studied. Ainslie et al. found that naively switching a model to MQA can hurt test perplexity a bit, but using GQA with $G$ around 8 or 16 recovers most of it [34] [35]. They even showed a technique to *uptrain* a pre-trained MHA model into an MQA/GQA model with only 5% of original compute [46], by merging the learned heads (e.g. by averaging their weights) [47] and then fine-tuning. This suggests it's possible to retrofit existing models with these efficient attention mechanisms.

**Implementation in code:** To implement MQA in a Transformer, one needs to change the projection for keys and values. Here's a snippet illustrating a modified multi-head attention forward with MQA (single shared KV) in PyTorch-like style:

```
class MultiHeadAttentionMQA(nn.Module):
    def __init__(self, d_model, n_heads, d_k):
        super().__init__()
        self.n_heads = n_heads
        self.d_k = d_k
        # Queries have n_heads separate projections, keys/values have 1
        self.W_q = nn.Linear(d_model, n_heads * d_k, bias=False)
        self.W_k = nn.Linear(d_model, d_k, bias=False)        # one shared key
head
        self.W_v = nn.Linear(d_model, d_k, bias=False)
# one shared value head
        self.W_o = nn.Linear(n_heads * d_k, d_model, bias=False)
    def forward(self, x, kv_cache=None):
        # x: [batch, seq, d_model]
        batch, seq_len, _ = x.size()
        # Project queries, keys, values
        Q = self.W_q(x).view(batch, seq_len, self.n_heads, self.d_k)
        K = self.W_k(x).view(batch, seq_len, 1, self.d_k)
        V = self.W_v(x).view(batch, seq_len, 1, self.d_k)
        # Transpose to shape [batch, n_heads, seq_len, d_k] for Q and same for
K,V
        Q = Q.transpose(1, 2)        # [batch, n_heads, S, d_k]
        K = K.transpose(1, 2)        # [batch, 1, S, d_k]
        V = V.transpose(1, 2)        # [batch, 1, S, d_k]
        # If using cache for autoregressive decoding, append new K,V to cache
from previous steps
        if kv_cache is not None:
            # kv_cache is a tuple of (K_cache, V_cache) from previous tokens
            K_prev, V_prev = kv_cache
            K = torch.cat([K_prev, K], dim=2)
            V = torch.cat([V_prev, V], dim=2)
        # In multi-query, duplicate K and V across heads for attention calc
        K_expanded = K.expand(batch, self.n_heads, K.size(2), self.d_k)
```

```
# copy along head dim
        V_expanded = V.expand(batch, self.n_heads, V.size(2), self.d_k)
        # Compute attention scores
        attn_scores = torch.matmul(Q, K_expanded.transpose(-2, -1))       #
[batch, n_heads, S, S]
        attn_scores = attn_scores / math.sqrt(self.d_k)
        # (Apply mask for causality if needed here)
        attn_weights = torch.softmax(attn_scores, dim=-1)
        # Attention output
        context = torch.matmul(attn_weights, V_expanded)                  #
[batch, n_heads, S, d_k]
        context = context.transpose(1, 2).contiguous().view(batch, seq_len,
self.n_heads * self.d_k)
        output = self.W_o(context)  # [batch, seq, d_model]
        return output, (K, V)       # return new KV cache for next step
```

In this code, $W^Q$ produces $n_{\text{heads}}$ different query projections, while $W^K, W^V$ each produce only one head's worth of keys/values (`d_k` each). We then expand that single head to all heads when computing scores. This is the simplest MQA; for GQA, one would produce `n_kv_heads = G` keys and values and expand each to a subset of heads (e.g. using `repeat_interleave` for each group). The rest of the attention computation remains the same.

**KV Cache integration:** The above also shows how a KV cache might be handled – if we're generating autoregressively, at each new token we append its K and V to the cache (which is of shape `[batch, 1 (or G), seq_so_far, d_k]` for MQA/GQA). In MQA, the cache size per layer is tiny (only 1 head of dimension $d_k$ per past token) compared to MHA (which would be $H$ heads per token). For example, a standard 16-head layer caching 8k tokens of dimension 64 each would store $16 \times 8000 \times 64 = 8,192,000$ floats; the MQA version stores $1 \times 8000 \times 64 = 512,000$ floats (32× smaller). This directly translates to less GPU memory usage and faster memory copying. As one source noted, in modern accelerators the **memory bandwidth** to load KV cache is often the decoder bottleneck, and MQA alleviates that greatly [48] [49].

**Conclusion on MQA/GQA:** These techniques are among the most impactful for long-context inference. They do involve a slight loss of modeling power (since heads have less independence), but in practice models like PaLM have shown that it's possible to train very high-quality LMs with multi-query attention [33]. GQA further sweetens the deal by giving back some head diversity for a fractional cost. If you are designing a Transformer for contexts of 8k, 16k tokens or more, using MQA/GQA is almost a necessity to keep inference tractable.

## 5. RMSNorm vs. LayerNorm: Faster Normalization

Transformers rely on normalization layers for stable training. The original Transformer used **Layer Normalization (LayerNorm)**, which normalizes each token's hidden state by subtracting the mean and dividing by the standard deviation of its features [10] [50]. While effective, LayerNorm can be somewhat expensive and introduces an additional bias parameter (the "shift" after scaling). A newer alternative, **RMSNorm** (Root Mean Square Norm), omits the mean subtraction and normalizes only by the root mean

square of features [10]. This was introduced by Zhang & Sennrich (2019) as a simplification of LayerNorm [9].

**LayerNorm recap:** Given a vector $x \in \mathbb{R}^d$ (the features of one token at one layer), LayerNorm computes $\mu = \frac{1}{d}\sum_i x_i$ and $\sigma^2 = \frac{1}{d}\sum_i (x_i - \mu)^2$. Then outputs $y_i = \frac{x_i - \mu}{\sqrt{\sigma^2 + \epsilon}} \gamma_i + \beta_i$, with learnable gain $\gamma$ and bias $\beta$. That centering ($- \mu$) and scaling by std ensure zero-mean, unit-variance output (modulo $\gamma,\beta$).

**RMSNorm:** In RMSNorm, we skip computing $\mu$ and just use the root mean square. The formula is:

$$ \text{RMSNorm}(x) = \frac{x}{\sqrt{\frac{1}{d}\sum_{i=1}^d x_i^2 + \epsilon}} \odot \gamma, $$

where $\gamma$ is a learnable weight vector and $\odot$ is elementwise multiplication [10] [50]. There is no $\beta$ (no bias term). In code, this is even simpler: compute `rms = torch.rsqrt(mean(x**2) + eps)`, then `x_norm = x * rms * weight` [51].

**Trade-offs:**

- *Computational efficiency:* RMSNorm avoids calculating the mean, which saves some operations. It also avoids subtracting the mean from every element. Especially for large hidden dimensions, this can shave off some time. As one source notes, RMSNorm requires **fewer calculations and has a smaller memory footprint** than LayerNorm [52] [53]. In big models, LayerNorm's cost is relatively small overall, but every bit counts. Some reports (e.g., from the LLaMA 2 team) suggest RMSNorm can be 10–30% faster in practice in those models, and simpler to implement in custom kernels.
- *Numerical behavior:* By not subtracting the mean, RMSNorm doesn't force the activations to be zero-mean. This means it doesn't "center" the distribution. In some cases, that can be fine or even beneficial (centering might not be crucial for transformers). RMSNorm is *less sensitive to outliers* because one large feature won't shift the whole mean, though it will still increase the RMS a bit [54]. On the flip side, **not centering can allow a drift** in the hidden state baseline. Empirically, researchers found RMSNorm yields similar or only slightly worse performance than LayerNorm, and in very large scale training it can be more stable (perhaps because it avoids some issues with zero-mean assumptions).
- *Memory usage:* Both are $O(d)$ per token, but RMSNorm has one less parameter tensor (no $\beta$) and slightly lower overhead.

**Use in LLMs:** The **LLaMA** models famously adopted Pre-Normalization using RMSNorm in each transformer block [55] [9]. They normalize at the input of sub-layers (instead of output, as GPT-2 did) and use RMSNorm rather than LayerNorm. The LLaMA paper cites Zhang & Sennrich (2019) and presumably found it effective [55] [9]. Many other modern LMs have followed suit. For instance, Hugging Face's Transformer implementations often have RMSNorm layers (there is even a `nn.RMSNorm` in PyTorch now for this).

Meta's rationale was likely partly **speed**: when you scale to 65B parameters, even small savings per layer add up. A Medium article summarizing LLaMA-2 mentions *"RMSNorm achieves comparable performance to LayerNorm with a 10–50% improvement in efficiency."* [56] [57] (This range might depend on model and hardware specifics, but it shows the perceived advantage.)

One potential drawback is, if a model heavily relied on the centering effect of LayerNorm to stabilize means, switching to RMSNorm could require retuning. But in practice, with residual connections in Transformers, the mean can be absorbed by subsequent layers and doesn't necessarily harm anything.

**Bottom line:** For implementing your own long-context Transformer, **using RMSNorm is a good default** unless you have a specific reason to prefer LayerNorm. It will slightly reduce compute and memory and has been battle-tested in recent models. As an example snippet, using PyTorch's built-in:

```
from torch.nn import RMSNorm, LayerNorm
norm1 = LayerNorm(normalized_shape=d_model)
norm2 = RMSNorm(dim=d_model)  # PyTorch RMSNorm expects argument name 'dim'
```

The outputs will have similar variance-stabilizing effects, but RMSNorm will not zero-mean the data. Empirically, models like GPT-3 used LayerNorm, while newer ones like LLaMA use RMSNorm – yet both achieve strong results, indicating RMSNorm is a viable replacement.

To summarize differences: **LayerNorm vs RMSNorm** [10] [53] :

- LayerNorm: subtracts mean and divides by std; two learnable params (scale & bias). Guarantees zero-mean, unit-variance per token (before scale/bias).
- RMSNorm: divides by RMS (equivalent to std if mean was zero); one learnable param (scale). Maintains original mean.
- RMSNorm is computationally lighter (no mean, no subtract) and slightly more memory-efficient. It may train just as well for large LMs, as evidenced by adoption in LLaMA and others. Some theoretical work has analyzed why removing mean can work – one hypothesis is that the residual connections anyway allow the model to adjust any mean shift.

One should note that in **pre-norm Transformer design** (normalizing at input of sublayers), RMSNorm can help avoid certain training instabilities that LayerNorm sometimes exhibits (LayerNorm can cause a scale ambiguity with the residual addition). Recent research showed that pre-norm + RMSNorm is mathematically equivalent in some cases to using a variant of LayerNorm, suggesting it's not fundamentally altering the function, just simplifying it [58] .

## 6. Key-Value Cache: Mechanics and Optimization

When generating text with a Transformer, a **key-value (KV) cache** is used to store the past context's keys and values for each attention layer. Instead of recomputing attention from scratch for all $n$ past tokens at each new step, the model reuses the keys and values from previous steps. This cache enables the Transformer to do incremental inference in $O(n)$ per token (for the attention part) instead of $O(n^2)$. However, as we've discussed, the KV cache can become enormous for long sequences and can strain memory and bandwidth.

**How KV caching works:** In a decoder-only Transformer (like GPT-style), each layer at time step $t$ produces a new key vector $K_t$ and value vector $V_t$ for the token generated at $t$. These are appended to an array of past keys $K_{\text{cache}} = [K_1, K_2, ..., K_{t-1}]$ (and similarly for values). At time $t$, when computing attention for layer $l$, you take the query $Q_t$ (for the current token) and compute

attention scores against all cached keys $[K_1..K_{t-1}]$. The weighted sum of all cached values gives the output. The cache grows with each token. By the end of generating an $N$-token sequence, you have stored $N$ keys and $N$ values per layer.

Without caching, one would have to recompute keys/values for all $N$ tokens at each step (re-quadratic cost). So caching **trades memory for compute**: we use extra memory to store them and extra memory bandwidth to fetch them, in order to avoid redundant computation.

**Memory trade-offs:** The cache memory per layer is (number of past tokens $N$) $\times$ (key size $d_k$ + value size $d_v$). For a typical model where $d_k = d_v = d_{\text{model}}/H$, the total for all heads is $N \times d_{\text{model}} \times 2$ (since keys and values each have that many floats). Multiply by number of layers $L$ to get total cache size. For long $N$, this becomes huge. In our earlier example, a 32-layer model with $d_{\text{model}}=4096$, $H=32$, at $N=8k$ tokens, the cache would be on the order of 7.8 GB * 32 layers ≈ 250 GB just for keys and values if using naive multi-head (which is clearly infeasible for a GPU) – hence techniques like MQA/GQA are *mandatory* at that scale.

In practice, even with MQA/GQA, caches can be tens of GB for the longest contexts (e.g., a 70B model at 100k tokens context might use ~80 GB of KV data). When the GPU memory isn't enough, systems resort to **offloading** cache to CPU memory or even disk (as done in Microsoft's DeepSpeed and in tools like Hugging Face's `accelerate` or Meta's `optimum`). Offloading, however, incurs latency due to limited PCIe or NVMe bandwidth [59] [60]. One paper noted: *"offloading KV cache to CPU/NVMe can still add non-trivial overhead to inference performance due to limited bandwidth."* [60]. So ideally, we keep as much of the cache on GPU as possible and reduce its size via architectural means (like MQA) or compression.

**Optimization techniques for KV cache:**

1. **Multi-Query/Grouped Attention:** As covered, this directly reduces cache size by sharing keys/values across heads. If you can use MQA or a small $G$, this is the simplest way to cut down memory by an $H$ or $H/G$ factor [39]. LLaMA-2's use of GQA was specifically to allow doubling context length to 4k without blowing up memory on their largest models.
2. **Cache precision:** Storing keys and values in half-precision (FP16 or even INT8) can cut memory in half or more. Many implementations use FP16 for KV cache with negligible quality loss, since attention dot-products don't require full FP32 precision.
3. **Segmented attention (sliding window):** Instead of attending to all $N$ tokens, some models restrict attention to a window of the most recent $w$ tokens (for very long contexts). This effectively discards or "forgets" old keys/values beyond the window, reducing cache length to $w$. Of course, this sacrifices the ability to attend to very old tokens, so it's a task-dependent heuristic. (This is not used in models like GPT-4 or Claude to our knowledge, but it is used in some realtime applications where only a moving window is needed.)
4. **Adaptive KV caching (FastGen):** A recent research direction is to **compress the KV cache** by removing entries that are not needed by the model's attention heads. "FastGen" (ICLR 2024) introduced an adaptive caching scheme that analyzes attention patterns and drops certain keys dynamically [61] [62]. For example, they found some heads only attend locally – so you don't need to keep very old tokens for those heads [63]. Other heads focus only on special tokens or punctuation – again you could prune irrelevant keys for those [64]. By combining strategies, they achieved about **20–40% memory reduction** in KV cache on LLaMA-1 models with minimal perplexity increase [65] [66]. Essentially, they compress the cache per head by dropping low-attention or unnecessary

tokens. This approach is quite advanced and requires either fine-tuning the model to tolerate missing keys or carefully engineering a method to decide drops on the fly. But it's promising for ultra-long contexts.

5. **Alternative architectures:** There are transformer variants (like Longformer, BigBird, Reformer, etc.) that use sparse or factorized attention to avoid full $N^2$ attention. These typically don't maintain a full KV cache either – e.g., Retrieval Transformers fetch relevant pieces of context instead of caching everything. However, those are beyond our current scope as they change the model's connectivity.

For a standard Transformer, the best immediate approach is using MQA/GQA and half precision. That alone can often enable 8k or 16k contexts on modern hardware if the model isn't too large. If more is needed, combining that with intelligent cache management (like discarding oldest tokens if you know the task doesn't need them) can help.

**Mechanics recap:** It's important to understand that the KV cache is **per layer**. So a 12-layer model will have 12 caches (one for each layer's keys/values). These are usually stored as PyTorch tensors of shape `[batch, n_heads, seq_len, d_k]` (and same for values). When using caching in code, one typically initializes an empty cache and at each generation step updates it. For example:

```
# Pseudocode for generating sequence with cache
K_cache = [None] * L  # list of length L for each layer
V_cache = [None] * L
for t in range(N):
    x_t = input_token[t] if t < input_length else generated_token  # get token
embedding
    for ℓ in range(L):
        # compute attn ℓ
        Q = linear_Qℓ(x_t)
        K = linear_Kℓ(x_t)  # shape [batch, d_k] (if MQA for simplicity)
        V = linear_Vℓ(x_t)  # shape [batch, d_v]
        if t == 0:
            K_cache[ℓ] = K.unsqueeze(2)  # [batch, 1, d_k]
            V_cache[ℓ] = V.unsqueeze(2)
        else:
            # append to existing cache on seq_len dim
            K_cache[ℓ] = torch.cat([K_cache[ℓ], K.unsqueeze(2)], dim=2)
            V_cache[ℓ] = torch.cat([V_cache[ℓ], V.unsqueeze(2)], dim=2)
        # Use Q and K_cache[ℓ] to compute attention for layer ℓ
        # ...
        x_t = output_of_layer_ℓ  # becomes input to next layer
    # final x_t is output of last layer for token t.
    if t >= input_length:
        generated_token = sample_from_logits(output_head(x_t))
```

This illustrates how caches grow. The operations like `torch.cat` are efficient, but one must be mindful of GPU memory fragmentation (preallocating a max-size tensor and slicing is often better).

To avoid a large concatenation cost at each step, some implementations allocate a fixed tensor for cache (e.g. [batch, n_heads, max_seq, d_k]) and write into it at index $t$. This prevents repeatedly copying data. For example, NVIDIA's FasterTransformer library and others do this.

In summary, the KV cache is both a **boon and a bane**: it makes auto-regressive inference feasible by reusing past computations, but it eats up memory proportionally to context length. Techniques like MQA/GQA attack the problem at the source by making each step lighter to store, and further research is ongoing to compress or intelligently manage caches.

For a self-study question: consider *why we don't cache queries*. The reason is queries are computed from the current token's hidden state and aren't reused – at next step, the next token will have its own new query. Only keys and values need caching because they represent information *from past tokens* that future queries will attend to.

## 7. Code Demo: Adding RoPE and MQA to a Transformer

To solidify understanding, let's walk through a simple Transformer block modification to include **Rotary Positional Embeddings (RoPE)** and **Multi-Query Attention (MQA)**. We will focus on the self-attention part of a decoder block.

Assume we have a Transformer block with an attention sublayer and a feed-forward sublayer. We want to:

- Integrate RoPE such that queries and keys are rotated by position before computing dot-products.
- Use MQA to share keys/values across heads.

Below is a simplified example in PyTorch-like pseudocode:

```
class TransformerBlockLongContext(nn.Module):
    def __init__(self, d_model, n_heads, d_k, d_ff):
        super().__init__()
        self.attn = MultiHeadAttentionMQA(d_model, n_heads, d_k)
        self.norm1 = RMSNorm(dim=d_model)                    # using RMSNorm
as discussed
        self.ff = nn.Sequential(
            nn.Linear(d_model, d_ff),
            nn.GELU(),
            nn.Linear(d_ff, d_model)
        )
        self.norm2 = RMSNorm(dim=d_model)
        # Prepare RoPE tables (cosines and sines) for maximum context length
        max_len = 8192
        self.register_buffer("cos_table", get_rope_cosines(max_len, d_k),
persistent=False)
        self.register_buffer("sin_table", get_rope_sines(max_len, d_k),
persistent=False)
    def forward(self, x, cache=None):
```

```
        """
        x: [batch, seq_len, d_model]
        cache: tuple of (K_cache, V_cache) from previous steps for attention, or
None if not using cache.
        """
        # Self-attention with RoPE and MQA
        residual = x
        qkv_out, new_cache = self.attn_with_rope(x, cache)     # custom method
integrating RoPE in attn
        x = residual + qkv_out                                 # add residual
connection
        x = self.norm1(x)
        # Feed-forward
        residual = x
        ff_out = self.ff(x)
        x = residual + ff_out
        x = self.norm2(x)
        return x, new_cache

    def attn_with_rope(self, x, cache):
        # Project queries, keys, values using MQA attn module
        # attn_output: [batch, seq, d_model], new_cache: (K, V) tuples
        # We need to intercept inside the MQA attention to apply RoPE before
score calc.
        batch, seq_len, _ = x.size()
        # Compute Q, K, V projections
        Q = self.attn.W_q(x).view(batch, seq_len, self.attn.n_heads,
self.attn.d_k)
        K = self.attn.W_k(x).view(batch, seq_len, 1, self.attn.d_k)
        V = self.attn.W_v(x).view(batch, seq_len, 1, self.attn.d_k)
        # Apply RoPE to Q and K
        Q = apply_rope(Q, self.cos_table, self.sin_table)      # rotate each
query vector
        K = apply_rope(K, self.cos_table, self.sin_table)
# rotate each key vector
        # Continue with attention computation (similar to
MultiHeadAttentionMQA.forward)
        Q = Q.transpose(1, 2)    # [batch, n_heads, S, d_k]
        K = K.transpose(1, 2)    # [batch, 1, S, d_k]
        V = V.transpose(1, 2)    # [batch, 1, S, d_k]
        if cache is not None:
            # Append to existing cache
            K_prev, V_prev = cache
            K = torch.cat([K_prev, K], dim=2)
            V = torch.cat([V_prev, V], dim=2)
        # Expand K, V for multi-query
        K_expand = K.expand(batch, self.attn.n_heads, K.size(2), self.attn.d_k)
        V_expand = V.expand(batch, self.attn.n_heads, V.size(2), self.attn.d_k)
```

```
        # Dot-product attention
        scores = torch.matmul(Q, K_expand.transpose(-2, -1)) /
math.sqrt(self.attn.d_k)
        # (Apply causal mask if needed)
        weights = torch.softmax(scores, dim=-1)
        attn_output = torch.matmul(weights, V_expand)          # [batch,
n_heads, S, d_k]
        attn_output = attn_output.transpose(1, 2).contiguous().view(batch,
seq_len, self.attn.n_heads * self.attn.d_k)
        # Output projection
        attn_output = self.attn.W_o(attn_output)
        new_cache = (K, V)
        return attn_output, new_cache
```

Let's break down what's happening in `attn_with_rope`:

- We compute $Q, K, V$ via the linear projections. Note $K, V$ have one head (MQA).
- We apply `apply_rope` to $Q$ and $K$ using precomputed tables of cosines/sines (`self.cos_table` and `self.sin_table`). These tables would be computed such that `cos_table[pos, :] = [\cos(pos \cdot \theta_1), \cos(pos \cdot \theta_2), ...]` for the rotation angles for each pair dimension. The function `apply_rope` is essentially the one defined in section 2.
- We then incorporate any existing cache by concatenating previous $K, V$ (for autoregressive generation). If not generating (e.g., doing full context in one forward pass), `cache` might be None and we skip that.
- We expand the single $K, V$ to all heads (`K_expand`, `V_expand`). This uses broadcasting for MQA.
- Compute attention scores, apply softmax, and combine with values as usual.
- The rest is standard: combine heads and apply output linear layer.

Using this block in either training (with full sequence input) or inference (with caching) would allow the model to benefit from RoPE's longer-range generalization and MQA's speed. In an actual training scenario, you'd likely not use the caching path (since you provide full sequences), and possibly you'd incorporate causal masking for the self-attention if it's a decoder.

**Verifying behavior:** One can test such a module by checking that:

- Outputs are the same dimension as inputs.
- RoPE doesn't change dimensions, just content – so one might test that disabling RoPE (e.g., using identity rotation) yields similar results to baseline, to ensure it's implemented right.
- The KV cache gets larger by exactly `seq_len` each time you call the block in inference mode.
- Quality: to truly test RoPE+MQA, you'd train a small model on a task and see if it can extrapolate to longer sequences.

This code example is of course a simplification. Real models have multi-head cross-attention (in encoder-decoder models) which could also benefit from RoPE (often applied in encoder-decoder as well) and sometimes use ALiBi instead or in addition. But it shows how relatively straightforward it is to integrate these concepts.

## 8. Benchmarking Inference at 8k Context Length

To appreciate the effect of these efficient design choices, let's consider inference performance at **8,192 tokens (8k)** – a length that is 4× the standard 2048 token context of original GPT-3. We compare two hypothetical model setups:

- **Baseline model:** 24-layer Transformer, $d_{\text{model}}=1024$, $H=16$ heads, using absolute positional embeddings and standard multi-head attention (no efficiency tweaks). Context = 8192.
- **Efficient model:** Same as above but using RoPE for positions and MQA (1 KV head) in all layers, and RMSNorm instead of LayerNorm.

**Latency:** The baseline model must perform 16-head attention over 8192 tokens. The attention score matrix per layer is $8192 \times 8192$ (roughly 67 million entries) per head – though in practice optimized kernels handle multi-head together. With caching, it still has to load 16 heads worth of keys/values (so 16 * 8192 * 64 numbers assuming $d_k=64$) each time. The efficient model with MQA loads 1 head worth (8192 * 64 numbers) – 16× fewer. This translates to a major reduction in memory bandwidth usage each step. In benchmarks, we would expect the efficient model to generate tokens significantly faster. Indeed, if we extrapolate from the Fireworks AI result (11× throughput with MQA for long seqs) [11] [12], at 8k length we might see on the order of ~8–10× more tokens per second.

For example, if the baseline model generates at, say, 5 tokens/second at 8k length, the MQA model might achieve ~40–50 tokens/sec on the same hardware. The **latency per token** also drops because the attention computation is lighter. Fireworks reported ~30% lower per-token latency with MQA [11]. In practice, the speed-up grows with sequence length: at very short sequences, MQA vs MHA difference is small (memory not a bottleneck), but at 8k it becomes big.

**Memory usage:** We can calculate approximate memory usage for the KV cache and activations:

- *Baseline:* KV cache per layer = $16 \text{ (heads)} \times 8192 \times d_k$. If $d_k = 64$, that's $16*8192*64 \approx 8.39 \times 10^6$ floats, which at 4 bytes each is ~33.6 MB per layer for keys (and same for values). So ~67 MB per layer. For 24 layers, ~1.6 GB just for the cache. Plus activations and temporary buffers, etc. Also, if using float16, it'd be half that (~0.8 GB).
- *Efficient:* KV cache per layer = $1 \times 8192 \times 64 = 524,288$ floats (~2.1 MB) per layer for keys (and 2.1 MB for values). So ~4.2 MB per layer. For 24 layers, ~100 MB total. This is an order of magnitude smaller. It easily fits in GPU memory even for large models. This illustrates why LLaMA-2's designers were comfortable extending context length when using GQA – the memory overhead was manageable.

It's not just memory: reading 1.6 GB of data from GPU memory every step vs 0.1 GB can be a huge difference in throughput, as GPU memory bandwidth might be ~900 GB/s on an A100, but if you are saturating that with KV loads, it slows other operations.

**Throughput experiment:** If we actually run a timed experiment for a single batch generation of 8k tokens on these two models (assuming same compute capability):

- The baseline may have to do 24 layers * (attention over 8k) for each new token. The complexity is roughly $24 * (8k * 8k)$ operations per token for attention = ~$24 * 64$ million = ~$1.5$ billion operations per token (plus feedforward etc.).
- The efficient model with MQA does $24 * (8k * 8k)$ but only 1 head's worth = ~1/16 of that for attention = ~$0.09$ billion ops per token for attention (plus a bit extra for rope rotations, which are negligible compared to 8k*8k).

So purely by ops count, we expect ~10–15× speedup. In reality, not all parts scale that cleanly (feed-forward layers still take same time in both), but attention often dominates at extreme lengths.

**Real benchmark data:** In lieu of our own run, we cite the result that *"optimizing for MQA can lead to 11× better throughput and 30% lower latency… compared to baselines without MQA"* [11] [12]. That result was on a system using LLaMA-2 and Falcon models (which also incorporate grouped-attention and other tricks). It confirms that these efficiency changes are not just theoretically nice but have **real-world impact** – enabling long-context models to run in production with reasonable speed.

**Quality at 8k:** It's worth mentioning that if a model is trained on 2k sequences but tested on 8k, techniques like RoPE and ALiBi help it not degrade. Without them, a naive model might show very high perplexity or generate incoherent text beyond 2k (due to lack of positional generalization). So the benefit is twofold: speed and the ability to even utilize 8k context meaningfully.

To summarize this benchmark: a Transformer with RoPE + MQA at 8192 tokens can achieve an order of magnitude faster inference throughput and use an order of magnitude less memory compared to a classical Transformer with absolute positions and multi-head attention. These numbers justify why practically all new large models incorporate these efficient design patterns when pushing context lengths higher.

## 9. Case Studies of Real Models

It's enlightening to see how the concepts we've discussed manifest in state-of-the-art models:

- **LLaMA (2023, Meta AI):** LLaMA-1 models (7B, 13B, 33B, 65B) were trained with a 2048 token context and made several efficiency-focused design choices. They **removed learned position embeddings in favor of RoPE** at each layer [13], enabling them to potentially extrapolate a bit beyond 2048. They also used **RMSNorm** instead of LayerNorm throughout [55] [9], and *pre-normalized* each sublayer (improving training stability for deep nets). LLaMA did *not* initially use multi-query attention – it had full multi-head attention (and indeed, the large LLaMA models had enormous memory footprints for cache). However, when **LLaMA-2** was released, Meta revealed that the 70B and 34B variants use **Grouped-Query Attention (GQA)** to reduce memory: specifically 8 key/value heads instead of 32 (for 70B) [42]. This means groups of 4 query heads share one KV – a compromise that saved memory bandwidth while keeping quality high. LLaMA-2 also extended context length to 4k tokens. Using RoPE made this straightforward – they didn't have to introduce a new position mechanism, they likely just trained a bit with longer sequences and relied on RoPE's relative encoding to handle it. In summary, LLaMA demonstrates the use of *RoPE + RMSNorm (+ GQA in v2)* in a production-grade

model [13] [9]. These contributed to LLaMA's efficiency, allowing a 13B model to rival a 175B GPT-3 on many tasks [67].

- **Claude (2023, Anthropic):** Claude is notable for its **100k token context window** in Claude 2 (and even 1M in a newer Claude variant). Although Anthropic hasn't published full technical details, they have indicated needing to **optimize the model and training** to achieve this. It's likely Claude uses **some form of rotary or ALiBi-like positional encoding** so it can handle extremely long sequences without position embedding overflow. Claude might also use **multi-query attention or a variant** – otherwise the memory for 100k tokens would be untenable. We do know Anthropic trained on long sequences and did special curricula to teach the model to use very long contexts effectively (including prompt techniques to fetch relevant info) [68] [69]. One can speculate that Claude employs *segment-wise attention* or compresses internal states (it might not attend fully densely to 100k tokens at every layer, because that's 100k×100k attention which is huge). But given they advertise a full 100k window, they probably rely on massive optimization and perhaps some sparsity. Claude is an extreme case illustrating that **with enough engineering, transformers can be pushed to read an entire book in one go**. Without RoPE/ALiBi, this would be impossible (learned embeddings for 100k positions are not feasible to train traditionally). So Claude likely builds on ideas from ALiBi (for length extrapolation) and uses efficient attention (maybe it shares keys/values or has special long-context decoder architectures). The fact that Claude retains good performance even at 100k context suggests careful training – it's not just architecture, but also ensuring the model learns to utilize distant information. Anthropic's own case study showed that even Claude can forget the earlier parts of a long prompt, and they experimented with prompting techniques to mitigate that [70] [69]. In any case, Claude stands as a testament that **long contexts are achievable** and will likely become more common.

- **PaLM (2022, Google):** PaLM was a 540B parameter model with 2048 context length. Notably, PaLM **used Multi-Query Attention** in its decoder layers [33]. This was mentioned in the GQA paper: *"some language models already use MQA, such as PaLM (Chowdhery et al., 2022), many do not, including … LLaMA"* [33]. Using MQA was critical for PaLM because at 540B, every efficiency gain counts (serving 540B with full multi-head attention would have been painfully slow and memory-heavy). PaLM also used a **rotary or relative position method**. The details in the paper are a bit subtle: PaLM's architecture description mentions **"relative positional encodings"** (it might have used a sine or RPE variant). It did *not* use ALiBi as far as known; ALiBi was introduced around the same time but not integrated in PaLM. However, PaLM's success with MQA paved the way for others. Google's next models (like PaLM 2) continue using those tricks and also incorporate new ideas like dual query layers, etc.

- **GPT-3 / GPT-4 (OpenAI):** GPT-3 (2020) didn't have these innovations – it used learned position embeddings (2048-length) and standard multi-head attention with LayerNorm. That's partly why GPT-3 needed 175B parameters to reach a certain level – it was a more "basic" Transformer. Over time, GPT-3's successors likely have adopted some of these improvements. GPT-4 (2023) is not fully public in architecture, but there are hints it uses better position encodings (perhaps rotary or a mix) and possibly some form of grouped attention to handle its context lengths (GPT-4 can handle 8k and 32k contexts). The 32k GPT-4 model presumably uses something like ALiBi or an advanced position embedding to generalize beyond training (since training on 32k would be very costly). OpenAI has not confirmed, but it would be unsurprising if GPT-4 uses a variant of **ALiBi** – since OpenAI

researchers were aware of that method. Also, for the 32k model, multi-query attention or at least a memory-optimized attention is very likely (otherwise the latency on user queries would be large).

- **Others:** Models like **Bloom** (176B, 2048 tokens) used rotary embeddings and multi-head attention (no MQA). **OPT** (Meta's model before LLaMA) used learned embeddings and no fancy attention – it struggled to extrapolate and wasn't as efficient. **T5** (Google, 2020) used a relative positional bias (like a learned ALiBi variant) and that helped it handle longer inputs than trained on, but T5 did not use MQA (it had fewer heads though). **Falcon** (TII UAE, 2023) is a 40B model that uses *multi-query attention* for the decoder, and they credit that as one reason it's fast. In fact, Falcon was known for being more efficient than other models of similar size, partly due to MQA. It also used rotary embeddings.

In summary, the trend in real-world LLM development has been to incorporate *multiple* of these efficiency techniques: nearly all use either RoPE or ALiBi for positioning (absolute position embeddings are mostly a thing of the past for new models), many are using RMSNorm for speed, and more are adopting MQA/GQA as context lengths increase. These innovations allow large models to be served in production at reasonable cost and allow researchers to explore **longer context windows** which unlock new applications (e.g., processing long documents, multi-step reasoning over extended text, etc.).

## 10. Summary and Takeaways

In this guide, we explored how to design Transformers for long contexts and fast inference, covering everything from positional encodings to attention architectures and normalization. The key takeaways are:

- **Positional encodings for extrapolation:** Traditional absolute positional embeddings don't generalize to longer inputs than seen in training. Techniques like **Rotary Positional Embeddings (RoPE)** [13] and **ALiBi (Attention with Linear Biases)** [20] enable models to handle longer sequences by encoding relative position information or biasing attention scores. RoPE rotates Q,K vectors to make attention inherently relative [14], while ALiBi adds a distance-based penalty encouraging recency [21]. Both are parameter-efficient and have become common in LLMs for >2k context. If you need a model to "train short, test long," consider ALiBi or RoPE to avoid zero-shot length generalization issues.
- **Efficient multi-head variants:** The memory bandwidth bottleneck in autoregressive inference can be addressed by **Multi-Query Attention (MQA)** and **Grouped-Query Attention (GQA)**. MQA shares one key/value across all heads [28], dramatically cutting down on KV cache size and improving decoding speed [8]. GQA generalizes this to a few shared keys/values per group of heads [38], preserving model quality better [34]. These modifications do not significantly increase model complexity or training cost (you can even convert a trained model to use them with some fine-tuning [47]). For long contexts, using MQA/GQA is practically necessary – as evidenced by LLaMA-2, PaLM, and others adopting them.
- **Normalization and stability: RMSNorm** is a simple replacement for LayerNorm that improves computational efficiency (no mean subtraction) [53]. It has been used in LLaMA, Falcon, and other recent models to help with training stability in very deep networks and to marginally speed up inference. It's a drop-in replacement in most cases – the main difference is the lack of bias term and not centering activations, which typically doesn't hurt quality significantly [54]. Use RMSNorm for large models where LayerNorm overhead isn't negligible.

- **KV cache management:** The KV cache is what makes Transformers viable for generation, but it scales linearly with sequence length and can overwhelm memory for long contexts. We discussed how using MQA/GQA reduces cache by sharing across heads, and how half-precision and selective eviction can further optimize it [3] [61]. One should design the model with cache size in mind – e.g., a 100k context model must have a strategy (like block sparse attention or extremely low head count for KV) to manage that. Always consider the **memory vs compute trade-off**: in some edge cases, recomputing parts of the context (trading compute for memory) might be worthwhile if memory is at a premium (some inference engines allow recompute of keys for far-away tokens instead of storing all of them).
- **Holistic design:** Efficient long-context transformers require combining multiple techniques. For instance, just adding RoPE might let a model *function* at 8k, but it could still be too slow without MQA. Or just adding MQA without a suitable positional scheme might make a model fast but it might not know how to utilize a long context it wasn't trained on. The *best results come from using these in tandem*: e.g., LLaMA-2 uses RoPE + GQA + RMSNorm together.

Finally, a note on **practical implications**: With these improvements, it's now feasible to build language models that handle very long documents (tens of thousands of tokens) which was infeasible a couple of years ago. This opens up new use cases: summarizing lengthy financial reports, analyzing code repositories, having extended dialogues without forgetting earlier context, etc. Engineers deploying such models must be mindful of the hardware limitations – for example, even with all optimizations, a 100k context on a 70B model might be slow (so throughput concerns mean you might not always want to use the max context unless necessary).

**Study Questions:**

To ensure understanding, here are some questions and exercises:

1. **RoPE Mechanics:** In your own words, explain how rotary position embedding encodes relative positions. What happens to the dot product of two vectors if both are rotated by the same angle? [14] Why does this property make RoPE encoding distance information?
2. **Implementing ALiBi:** Sketch how you would modify the Transformer attention score calculation to include ALiBi biases [20]. What data structure would you use to represent the biases for an input of length $N$? How do the biases affect attention probabilities for distant tokens?
3. **MQA vs. GQA trade-offs:** Suppose a model with 16 heads originally has perplexity X. After converting it to MQA (1 head), perplexity becomes $X+\Delta$. How might using GQA with 4 groups (4 KV heads) compare in terms of perplexity and memory bandwidth? [38] Discuss why grouped attention might recover quality.
4. **RMSNorm Calculation:** Take a tiny example vector $x = [2, -2, 1, -1]$. Compute LayerNorm and RMSNorm for this vector (manually, with $\epsilon=0$ for simplicity). Which one changes the sign of some components and which doesn't? [10] What does that tell you about the effect of centering?
5. **KV Cache Growth:** If you have a generation of 1000 tokens with a model of 24 layers and 8 heads of dimension 64, how many float values are stored in the KV cache by the end (assume MHA)? How would this change if using MQA? (Show the calculation). What challenges might arise when trying to serve a batch of 10 such generations in parallel on a GPU with 12 GB memory?
6. **Benchmark Reasoning:** If an 8k context model with MHA runs at 1 token/s, and you switch to MQA and observe 5 tokens/s, what are some factors (aside from just memory bandwidth) that could limit

you from reaching the theoretical 16× speedup? Think about other parts of the transformer pipeline (e.g., feed-forward layers, data transfer, etc.).

7. **Real-world Reflection:** Find a recent open-source model (or a paper) that explicitly mentions using one of these techniques (RoPE, ALiBi, MQA, RMSNorm). Summarize how they justify its use. For example, how does the Falcon model justify multi-query attention in its release blog? [44]

Working through these questions will reinforce your grasp of efficient transformer design. As context lengths and model sizes continue to grow, mastering these techniques will be crucial for anyone training or deploying advanced language models. Good luck, and happy modeling!

---


[1] [2] [48] [1911.02150] Fast Transformer Decoding: One Write-Head is All You Need
https://ar5iv.labs.arxiv.org/html/1911.02150

[3] [4] [8] [31] [44] [49] [59] [60] [61] [62] [63] [64] [65] [66] Accelerating LLM Inference with efficient Self-Attention — MQA, GQA, MLA(DeepSeek), Adaptive KV Caching | by Sauradeep Debnath | GoPenAI
https://blog.gopenai.com/accelerating-llm-inference-with-efficient-self-attention-mqa-gqa-mla-deepseek-adaptive-kv-2bb48c14e798?gi=720e4101b0d7

[5] [6] [20] [21] [22] [23] [24] [25] [26] [27] [2108.12409] Train Short, Test Long: Attention with Linear Biases Enables Input Length Extrapolation
https://ar5iv.labs.arxiv.org/html/2108.12409

[7] [28] [32] [1911.02150] Fast Transformer Decoding: One Write-Head is All You Need
https://arxiv.org/abs/1911.02150

[9] [13] [18] [19] [55] [58] [67] [2302.13971] LLaMA: Open and Efficient Foundation Language Models
https://ar5iv.labs.arxiv.org/html/2302.13971

[10] [50] [51] [52] [53] [54] LayerNorm and RMS Norm in Transformer Models - MachineLearningMastery.com
https://machinelearningmastery.com/layernorm-and-rms-norm-in-transformer-models/

[11] [12] [45] Multi-Query Attention is All You Need
https://fireworks.ai/blog/multi-query-attention-is-all-you-need

[14] [15] [16] [17] Day 8/50: Building a Small Language Model from Scratch: What are Rotary Positional Embeddings (RoPE) | by Prashant Lakhera | Jul, 2025 | Medium
https://devopslearning.medium.com/day-8-50-building-a-small-language-model-from-scratch-code-positional-embeddings-53099f9f40bf

[29] [30] [33] [36] [37] [38] [47] [2305.13245] GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints
https://ar5iv.labs.arxiv.org/html/2305.13245

[34] [35] [40] [41] [46] [2305.13245] GQA: Training Generalized Multi-Query Transformer Models from Multi-Head Checkpoints
https://arxiv.org/abs/2305.13245

[39] Grouped-Query Attention · Issue #384 · meta-llama/llama - GitHub
https://github.com/facebookresearch/llama/issues/384

[42] Grouped-Query Attention · Issue #384 · meta-llama/llama · GitHub
https://github.com/meta-llama/llama/issues/384

43 Beyond Uniform Query Distribution: Key-Driven Grouped ... - arXiv
https://arxiv.org/html/2408.08454v1

56 apollo000104/LLAMA - GitHub
https://github.com/apollo000104/LLAMA

57 Llama From Scratch with Pytorch Lightning - After Hours Research
https://afterhoursresearch.hashnode.dev/llama-from-scratch-with-pytorch-lightning

68 69 70 Anthropic Shares Techniques to Optimize Claude's 100K Context Window for Better Recall
https://www.maginative.com/article/anthropic-shares-techniques-to-optimize-claudes-100k-context-window-for-better-recall/